Promise

A lightweight implementation of CommonJS Promises/A for PHP.

Table of Contents

Introduction

Promise is a library implementing CommonJS Promises/A for PHP.

It also provides several other useful promise-related concepts, such as joining multiple promises and mapping and reducing collections of promises.

If you've never heard about promises before, read this first.

Concepts

Deferred

A **Deferred** represents a computation or unit of work that may not have completed yet. Typically (but not always), that computation will be something that executes asynchronously and completes at some point in the future.

Promise

While a deferred represents the computation itself, a **Promise** represents the result of that computation. Thus, each deferred has a promise that acts as a placeholder for its actual result.

API

Deferred

A deferred represents an operation whose resolution is pending. It has separate promise and resolver parts.

```php
$deferred = new React\Promise\Deferred();

$promise = $deferred->promise();

$deferred->resolve(mixed $value);
$deferred->reject(\Throwable $reason);
```

The `promise` method returns the promise of the deferred.

The `resolve` and `reject` methods control the state of the deferred.

The constructor of the `Deferred` accepts an optional `$canceller` argument. See Promise for more information.

Deferred::promise()

```php
$promise = $deferred->promise();
```

Returns the promise of the deferred, which you can hand out to others while keeping the authority to modify its state to yourself.

Deferred::resolve()

```php
$deferred->resolve(mixed $value);
```

Resolves the promise returned by `promise()`. All consumers are notified by having `$onFulfilled` (which they registered via `$promise->then()`) called with `$value`.

If `$value` itself is a promise, the promise will transition to the state of this promise once it is resolved.

See also the `resolve()` function.

Deferred::reject()

```php
$deferred->reject(\Throwable $reason);
```

Rejects the promise returned by `promise()`, signalling that the deferred's computation failed. All consumers are notified by having `$onRejected` (which they registered via `$promise->then()`) called with `$reason`.

See also the `reject()` function.

PromiseInterface

The promise interface provides the common interface for all promise implementations. See Promise for the only public implementation exposed by this package.

A promise represents an eventual outcome, which is either fulfillment (success) and an associated value, or rejection (failure) and an associated reason.

Once in the fulfilled or rejected state, a promise becomes immutable. Neither its state nor its result (or error) can be modified.

PromiseInterface::then()

```php
$transformedPromise = $promise->then(callable $onFulfilled = null, callable $onRejected = nu
```

Transforms a promise's value by applying a function to the promise's fulfillment or rejection value. Returns a new promise for the transformed result.

The `then()` method registers new fulfilled and rejection handlers with a promise (all parameters are optional):

- `$onFulfilled` will be invoked once the promise is fulfilled and passed the result as the first argument.
- `$onRejected` will be invoked once the promise is rejected and passed the reason as the first argument.

It returns a new promise that will fulfill with the return value of either `$onFulfilled` or `$onRejected`, whichever is called, or will reject with the thrown exception if either throws.

A promise makes the following guarantees about handlers registered in the same call to `then()`:

1. Only one of `$onFulfilled` or `$onRejected` will be called, never both.
2. `$onFulfilled` and `$onRejected` will never be called more than once.

See also

- resolve() - Creating a resolved promise
- reject() - Creating a rejected promise

PromiseInterface::catch()

```php
$promise->catch(callable $onRejected);
```

Registers a rejection handler for promise. It is a shortcut for:

```php
$promise->then(null, $onRejected);
```

Additionally, you can type hint the `$reason` argument of `$onRejected` to catch only specific errors.

```php
$promise
    ->catch(function (\RuntimeException $reason) {
        // Only catch \RuntimeException instances
        // All other types of errors will propagate automatically
    })
    ->catch(function (\Throwable $reason) {
        // Catch other errors
    });
```

PromiseInterface::finally()

```php
$newPromise = $promise->finally(callable $onFulfilledOrRejected);
```

Allows you to execute "cleanup" type tasks in a promise chain.

It arranges for `$onFulfilledOrRejected` to be called, with no arguments, when the promise is either fulfilled or rejected.

- If `$promise` fulfills, and `$onFulfilledOrRejected` returns successfully, `$newPromise` will fulfill with the same value as `$promise`.
- If `$promise` fulfills, and `$onFulfilledOrRejected` throws or returns a rejected promise, `$newPromise` will reject with the thrown exception or rejected promise's reason.
- If `$promise` rejects, and `$onFulfilledOrRejected` returns successfully, `$newPromise` will reject with the same reason as `$promise`.
- If `$promise` rejects, and `$onFulfilledOrRejected` throws or returns a rejected promise, `$newPromise` will reject with the thrown exception or rejected promise's reason.

`finally()` behaves similarly to the synchronous finally statement. When combined with `catch()`, `finally()` allows you to write code that is similar to the familiar synchronous catch/finally pair.

Consider the following synchronous code:

```
try {
    return doSomething();
} catch (\Throwable $e) {
    return handleError($e);
} finally {
    cleanup();
}
```

Similar asynchronous code (with `doSomething()` that returns a promise) can be written:

```
return doSomething()
    ->catch('handleError')
    ->finally('cleanup');
```

PromiseInterface::cancel()

```
$promise->cancel();
```

The `cancel()` method notifies the creator of the promise that there is no further interest in the results of the operation.

Once a promise is settled (either fulfilled or rejected), calling `cancel()` on a promise has no effect.

~~PromiseInterface::otherwise()~~

Deprecated since v3.0.0, see `catch()` instead.

The `otherwise()` method registers a rejection handler for a promise.

This method continues to exist only for BC reasons and to ease upgrading between versions. It is an alias for:

```
$promise->catch($onRejected);
```

~~PromiseInterface::always()~~

Deprecated since v3.0.0, see `finally()` instead.

The `always()` method allows you to execute "cleanup" type tasks in a promise chain.

This method continues to exist only for BC reasons and to ease upgrading between versions. It is an alias for:

```
$promise->finally($onFulfilledOrRejected);
```

Promise

Creates a promise whose state is controlled by the functions passed to `$resolver`.

```php
$resolver = function (callable $resolve, callable $reject) {
    // Do some work, possibly asynchronously, and then
    // resolve or reject.

    $resolve($awesomeResult);
    // or throw new Exception('Promise rejected');
    // or $resolve($anotherPromise);
    // or $reject($nastyError);
};

$canceller = function () {
    // Cancel/abort any running operations like network connections, streams etc.

    // Reject promise by throwing an exception
    throw new Exception('Promise cancelled');
};

$promise = new React\Promise\Promise($resolver, $canceller);
```

The promise constructor receives a resolver function and an optional canceller function which both will be called with two arguments:

- `$resolve($value)` - Primary function that seals the fate of the returned promise. Accepts either a non-promise value, or another promise. When called with a non-promise value, fulfills promise with that value. When called with another promise, e.g. `$resolve($otherPromise)`, promise's fate will be equivalent to that of `$otherPromise`.
- `$reject($reason)` - Function that rejects the promise. It is recommended to just throw an exception instead of using `$reject()`.

If the resolver or canceller throw an exception, the promise will be rejected with that thrown exception as the rejection reason.

The resolver function will be called immediately, the canceller function only once all consumers called the `cancel()` method of the promise.

Functions

Useful functions for creating and joining collections of promises.

All functions working on promise collections (like `all()`, `race()`, etc.) support cancellation. This means, if you call `cancel()` on the returned promise, all promises in the collection are cancelled.

resolve()

```
$promise = React\Promise\resolve(mixed $promiseOrValue);
```

Creates a promise for the supplied `$promiseOrValue`.

If `$promiseOrValue` is a value, it will be the resolution value of the returned promise.

If `$promiseOrValue` is a thenable (any object that provides a `then()` method), a trusted promise that follows the state of the thenable is returned.

If `$promiseOrValue` is a promise, it will be returned as is.

The resulting `$promise` implements the `PromiseInterface` and can be consumed like any other promise:

```
$promise = React\Promise\resolve(42);

$promise->then(function (int $result): void {
    var_dump($result);
}, function (\Throwable $e): void {
    echo 'Error: ' . $e->getMessage() . PHP_EOL;
});
```

reject()

```
$promise = React\Promise\reject(\Throwable $reason);
```

Creates a rejected promise for the supplied `$reason`.

Note that the `\Throwable` interface introduced in PHP 7 covers both user land `\Exception`'s and `\Error` internal PHP errors. By enforcing `\Throwable` as reason to reject a promise, any language error or user land exception can be used to reject a promise.

The resulting `$promise` implements the `PromiseInterface` and can be consumed like any other promise:

```
$promise = React\Promise\reject(new RuntimeException('Request failed'));

$promise->then(function (int $result): void {
    var_dump($result);
}, function (\Throwable $e): void {
    echo 'Error: ' . $e->getMessage() . PHP_EOL;
});
```

Note that rejected promises should always be handled similar to how any exceptions should always be caught in a `try` + `catch` block. If you remove the last reference to a rejected promise that has not been handled, it will report an unhandled promise rejection:

```
function incorrect(): int
{
```

```
    $promise = React\Promise\reject(new RuntimeException('Request failed'));

    // Commented out: No rejection handler registered here.
    // $promise->then(null, function (\Throwable $e): void { /* ignore */ });

    // Returning from a function will remove all local variable references, hence why
    // this will report an unhandled promise rejection here.
    return 42;
}

// Calling this function will log an error message plus its stack trace:
// Unhandled promise rejection with RuntimeException: Request failed in example.php:10
incorrect();
```

A rejected promise will be considered "handled" if you catch the rejection reason with either the `then()` method, the `catch()` method, or the `finally()` method. Note that each of these methods return a new promise that may again be rejected if you re-throw an exception.

A rejected promise will also be considered "handled" if you abort the operation with the `cancel()` method (which in turn would usually reject the promise if it is still pending).

See also the `set_rejection_handler()` function.

all()

```
$promise = React\Promise\all(iterable $promisesOrValues);
```

Returns a promise that will resolve only once all the items in `$promisesOrValues` have resolved. The resolution value of the returned promise will be an array containing the resolution values of each of the items in `$promisesOrValues`.

race()

```
$promise = React\Promise\race(iterable $promisesOrValues);
```

Initiates a competitive race that allows one winner. Returns a promise which is resolved in the same way the first settled promise resolves.

The returned promise will become **infinitely pending** if `$promisesOrValues` contains 0 items.

any()

```
$promise = React\Promise\any(iterable $promisesOrValues);
```

Returns a promise that will resolve when any one of the items in `$promisesOrValues` resolves. The resolution value of the returned promise will be the resolution value of the triggering item.

The returned promise will only reject if *all* items in `$promisesOrValues` are rejected. The rejection value will be a `React\Promise\Exception\CompositeException` which holds all rejection reasons. The rejection reasons can be obtained with `CompositeException::getThrowables()`.

The returned promise will also reject with a `React\Promise\Exception\LengthException` if `$promisesOrValues` contains 0 items.

set_rejection_handler()

```
React\Promise\set_rejection_handler(?callable $callback): ?callable;
```

Sets the global rejection handler for unhandled promise rejections.

Note that rejected promises should always be handled similar to how any exceptions should always be caught in a `try` + `catch` block. If you remove the last reference to a rejected promise that has not been handled, it will report an unhandled promise rejection. See also the `reject()` function for more details.

The `?callable $callback` argument MUST be a valid callback function that accepts a single `Throwable` argument or a `null` value to restore the default promise rejection handler. The return value of the callback function will be ignored and has no effect, so you SHOULD return a `void` value. The callback function MUST NOT throw or the program will be terminated with a fatal error.

The function returns the previous rejection handler or `null` if using the default promise rejection handler.

The default promise rejection handler will log an error message plus its stack trace:

```
// Unhandled promise rejection with RuntimeException: Unhandled in example.php:2
React\Promise\reject(new RuntimeException('Unhandled'));
```

The promise rejection handler may be used to use customize the log message or write to custom log targets. As a rule of thumb, this function should only be used as a last resort and promise rejections are best handled with either the `then()` method, the `catch()` method, or the `finally()` method. See also the `reject()` function for more details.

Examples

How to use Deferred

```
function getAwesomeResultPromise()
{
    $deferred = new React\Promise\Deferred();

    // Execute a Node.js-style function using the callback pattern
    computeAwesomeResultAsynchronously(function (\Throwable $error, $result) use ($deferred)
```

```
        if ($error) {
            $deferred->reject($error);
        } else {
            $deferred->resolve($result);
        }
    });

    // Return the promise
    return $deferred->promise();
}

getAwesomeResultPromise()
    ->then(
        function ($value) {
            // Deferred resolved, do something with $value
        },
        function (\Throwable $reason) {
            // Deferred rejected, do something with $reason
        }
    );
```

How promise forwarding works

A few simple examples to show how the mechanics of Promises/A forwarding works. These examples are contrived, of course, and in real usage, promise chains will typically be spread across several function calls, or even several levels of your application architecture.

Resolution forwarding Resolved promises forward resolution values to the next promise. The first promise, `$deferred->promise()`, will resolve with the value passed to `$deferred->resolve()` below.

Each call to `then()` returns a new promise that will resolve with the return value of the previous handler. This creates a promise "pipeline".

```
$deferred = new React\Promise\Deferred();

$deferred->promise()
    ->then(function ($x) {
        // $x will be the value passed to $deferred->resolve() below
        // and returns a *new promise* for $x + 1
        return $x + 1;
    })
    ->then(function ($x) {
        // $x === 2
        // This handler receives the return value of the
        // previous handler.
```

```
        return $x + 1;
    })
    ->then(function ($x) {
        // $x === 3
        // This handler receives the return value of the
        // previous handler.
        return $x + 1;
    })
    ->then(function ($x) {
        // $x === 4
        // This handler receives the return value of the
        // previous handler.
        echo 'Resolve ' . $x;
    });

$deferred->resolve(1); // Prints "Resolve 4"
```

Rejection forwarding Rejected promises behave similarly, and also work similarly to try/catch: When you catch an exception, you must rethrow for it to propagate.

Similarly, when you handle a rejected promise, to propagate the rejection, "rethrow" it by either returning a rejected promise, or actually throwing (since promise translates thrown exceptions into rejections)

```
$deferred = new React\Promise\Deferred();

$deferred->promise()
    ->then(function ($x) {
        throw new \Exception($x + 1);
    })
    ->catch(function (\Exception $x) {
        // Propagate the rejection
        throw $x;
    })
    ->catch(function (\Exception $x) {
        // Can also propagate by returning another rejection
        return React\Promise\reject(
            new \Exception($x->getMessage() + 1)
        );
    })
    ->catch(function ($x) {
        echo 'Reject ' . $x->getMessage(); // 3
    });

$deferred->resolve(1);  // Prints "Reject 3"
```

Mixed resolution and rejection forwarding Just like try/catch, you can choose to propagate or not. Mixing resolutions and rejections will still forward handler results in a predictable way.

```php
$deferred = new React\Promise\Deferred();

$deferred->promise()
    ->then(function ($x) {
        return $x + 1;
    })
    ->then(function ($x) {
        throw new \Exception($x + 1);
    })
    ->catch(function (\Exception $x) {
        // Handle the rejection, and don't propagate.
        // This is like catch without a rethrow
        return $x->getMessage() + 1;
    })
    ->then(function ($x) {
        echo 'Mixed ' . $x; // 4
    });

$deferred->resolve(1);  // Prints "Mixed 4"
```

Install

The recommended way to install this library is through Composer. New to Composer?

This project follows SemVer. This will install the latest supported version from this branch:

```
composer require react/promise:^3.2
```

See also the CHANGELOG for details about version upgrades.

This project aims to run on any platform and thus does not require any PHP extensions and supports running on PHP 7.1 through current PHP 8+. It's *highly recommended to use the latest supported PHP version* for this project.

We're committed to providing long-term support (LTS) options and to provide a smooth upgrade path. If you're using an older PHP version, you may use the `2.x` branch (PHP 5.4+) or `1.x` branch (PHP 5.3+) which both provide a compatible API but do not take advantage of newer language features. You may target multiple versions at the same time to support a wider range of PHP versions like this:

```
composer require "react/promise:^3 || ^2 || ^1"
```

Tests

To run the test suite, you first need to clone this repo and then install all dependencies through Composer:

```
composer install
```

To run the test suite, go to the project root and run:

```
vendor/bin/phpunit
```

On top of this, we use PHPStan on max level to ensure type safety across the project:

```
vendor/bin/phpstan
```

Credits

Promise is a port of when.js by Brian Cavalier.

Also, large parts of the documentation have been ported from the when.js Wiki and the API docs.

License

Released under the MIT license.